

SkinSense — Complete Test Suite Documentation

188 tests · 10 test files · Express backend · React frontend · Python inference service

Categories: [Success Story](#) | [Error Handling](#) | [Edge Case](#)

Test Coverage Summary

File Name	Total Tests	Success Story	Error Handling	Edge Cases
auth.test.js	30	11	11	8
middleware.test.js	17	7	7	3
screening.test.js	27	10	13	4
useScreening.test.jsx	21	8	7	6
useVoiceRecorder.test.jsx	15	5	2	3
imageValidation.test.js	23	9	7	7
test_predict.py	19	9	4	6
test_transcribe.py	9	0	4	5
test_heatmap.py	10	4	2	4
test_utils.py	17	12	1	4
TOTAL — 188 Tests	188	75	58	55

SERVER — Express.js Backend (Jest + Supertest)

auth.test.js — 30 Tests · Authentication & User Management

#	Test Name	Description	Category
1	registers a patient successfully	POSTs valid username, password and role:'patient'. Expects 201 with a JWT token and user.role === 'patient' in the response body.	Success Story
2	registers a clinician successfully	POSTs valid credentials with role:'clinician'. Expects 201 with a token and user.role === 'clinician' confirming role is persisted.	Success Story
3	register with minimal required fields only	Sends only the three required fields (username, password, role) with no optional extras. Expects 201, confirming optional fields are truly optional.	Success Story
4	duplicate username returns 409	Mocks the DB to return an existing row for the username. Expects 409 Conflict so the frontend can prompt the user to choose a different name.	Error Handling
5	missing username field returns 400	Omits the username field from the request body. Expects 400 from express-validator before the controller is reached.	Error Handling
6	missing password field returns 400	Omits the password field. Expects 400 so the server never tries to hash an undefined value.	Error Handling
7	missing role field returns 400	Omits the role field. Expects 400 because role is required to assign correct permissions in the database.	Error Handling
8	invalid role value returns 400	Sends role:'admin' which is not in the allowed enum {patient, clinician}. Expects 400 to prevent privilege escalation.	Error Handling
9	DB insert error returns 500	Mocks pg.Pool.query to throw on the INSERT statement. Expects 500 and verifies no token is included in the error response.	Error Handling
10	response does not expose password_hash	Checks that the JSON response body has no password_hash field, ensuring hashed credentials are never sent to the client.	Edge Case
11	login with valid credentials returns 200	Mocks bcrypt.compare returning true and a matching DB user row. Expects 200 and a signed JWT token in the response body.	Success Story
12	returned JWT contains correct user id	Decodes the JWT from the login response and asserts the 'id' claim matches the mocked user's database id.	Success Story
13	returned JWT contains correct role claim	Decodes the JWT and asserts the 'role' claim matches the role stored in the mocked DB row.	Success Story
14	wrong username returns 401	Mocks an empty DB result for the username lookup. Expects 401 with a generic 'invalid credentials' message.	Error Handling
15	wrong password returns 401	Mocks bcrypt.compare returning false. Expects 401 with the same generic message as wrong username to prevent field enumeration.	Error Handling
16	wrong-username and wrong-password messages are identical	Compares the exact message string from both 401 scenarios. They must be identical so an attacker cannot determine which field was incorrect.	Edge Case
17	missing username in login body returns 400	Sends login body without username. Expects 400 from validation middleware before bcrypt is ever called.	Error Handling
18	missing password in login body returns 400	Sends login body without password. Expects 400 from validation middleware.	Error Handling
19	DB error during login returns 500	Mocks pg.Pool.query to throw during the SELECT. Expects 500 and no token in the body.	Error Handling
20	completely empty login body returns 400	Sends an empty JSON object {}. Expects 400 to guard against malformed automation requests or misconfigured clients.	Edge Case
21	logout blacklists the token	POSTs logout with a valid Bearer token. Expects 200 and verifies the token is added to the in-memory blacklist.	Success Story
22	re-use of blacklisted token returns 401	Immediately reuses the same token after logout. Expects 401 with a 'token revoked' message, enforcing session termination.	Error Handling

#	Test Name	Description	Category
23	logout without token returns 401	Calls POST /logout with no Authorization header. Expects 401 from the authenticate middleware before the controller runs.	Error Handling
24	logout with expired token returns 401	Signs a token with expiresIn: '-1s' and attempts logout. Expects 401 because expired tokens are invalid regardless of endpoint.	Edge Case
25	GET /profile returns patient data	Sends a valid patient JWT. Mocks DB to return a patient row. Expects 200 with username and role in the response.	Success Story
26	GET /profile returns clinician data	Sends a valid clinician JWT. Expects 200 with role: 'clinician', confirming role-specific data is returned correctly.	Success Story
27	profile with no DB row returns 404	Mocks the DB to return an empty rows array for the user lookup. Expects 404 Not Found.	Error Handling
28	expired token on profile returns 401	Signs a JWT with expiresIn: '-1s'. Expects 401 from authenticate middleware before the controller runs.	Error Handling
29	tampered token on profile returns 401	Modifies the payload section of a valid JWT then re-encodes it. Expects 401 because the signature no longer matches the secret.	Error Handling
30	no Authorization header on profile returns 401	Sends GET /profile with no headers. Expects 401, confirming the route has no anonymous fallback.	Edge Case

middleware.test.js — 17 Tests · JWT Auth, Role Guard, File Filters

#	Test Name	Description	Category
1	valid token passes authenticate middleware	Attaches a correctly signed JWT. Middleware calls next() and populates req.user with {id, role}.	Success Story
2	missing Authorization header returns 401	No Authorization header at all. Middleware returns 401 immediately without calling next().	Error Handling
3	wrong scheme (Basic not Bearer) returns 401	Sends 'Basic <token>'. Middleware rejects because it only accepts the Bearer scheme.	Error Handling
4	wrong JWT secret returns 401	Token signed with a different secret. jwt.verify throws JsonWebTokenError and middleware returns 401.	Error Handling
5	garbage token string returns 401	Sends a random non-JWT string after 'Bearer '. jwt.verify fails and middleware returns 401.	Error Handling
6	expired token returns 401	Token with expiresIn: '-1s'. Middleware catches TokenExpiredError and returns 401 with an expiry-specific error message.	Edge Case
7	blacklisted token returns 401	Token that appears in the blacklist (after logout). Middleware checks the blacklist first and returns 401 before verifying the signature.	Edge Case
8	clinician token on patient-only route returns 403	Attaches a clinician JWT to a route guarded by requireRole('patient'). Middleware returns 403 Forbidden.	Error Handling
9	patient token on clinician-only route returns 403	Attaches a patient JWT to a route guarded by requireRole('clinician'). Middleware returns 403 Forbidden.	Error Handling
10	matching role passes through to controller	Attaches a JWT whose role matches the route's required role. Middleware calls next() without error.	Success Story
11	JPEG MIME type accepted by image filter	Passes image/jpeg to the multer fileFilter callback. Callback receives null error and true, allowing the upload to proceed.	Success Story
12	PNG MIME type accepted by image filter	Passes image/png. Callback accepts it, confirming both standard image formats are whitelisted.	Success Story
13	GIF MIME type rejected by image filter	Passes image/gif. Callback receives an Error, confirming animated images are blocked before they reach disk.	Error Handling

#	Test Name	Description	Category
14	PDF MIME type rejected by image filter	Passes application/pdf. Callback receives an Error, confirming documents cannot be uploaded as skin images.	Error Handling
15	WAV MIME type accepted by audio filter	Passes audio/wav to the audio fileFilter. Callback accepts it as a valid format for the voice transcription pipeline.	Success Story
16	audio/webm;codecs=opus accepted by audio filter	Chrome records audio as 'audio/webm;codecs=opus'. The filter uses startsWith('audio/') so this compound MIME type is accepted correctly.	Edge Case
17	image/png rejected by audio filter	Passes image/png to the audio filter. Callback rejects it, preventing image files from being forwarded to the transcription service.	Error Handling

screening.test.js — 27 Tests · Screening Pipeline Endpoints

#	Test Name	Description	Category
1	POST /create returns new case with id	Authenticated patient POSTs to /create. DB mock returns a new row. Expects 201 and a case id in the response body.	Success Story
2	POST /create without auth token returns 401	No Authorization header. authenticate middleware blocks the request before the controller is reached.	Error Handling
3	JPEG upload stores file and returns path	Attaches a JPEG file. Multer stores it. Controller updates image_path in DB. Expects 200 with the stored file path.	Success Story
4	PNG upload accepted	Attaches a PNG file and follows the same success path as JPEG. Confirms both supported formats work end-to-end.	Success Story
5	PDF upload rejected returns 500	Attaches a PDF file. Multer's fileFilter throws 'Unsupported image format'. The global error handler returns 500.	Error Handling
6	no image file attached returns 400	No multipart body sent. Controller detects req.file is undefined and returns 400 'No image file provided'.	Error Handling
7	ASR fallback when Python service unreachable	Python service is not running (ECONNREFUSED). Controller catches the error and saves transcript_text:null, confirming graceful degradation.	Success Story
8	Chrome webm;codecs=opus audio accepted	Sends audio/webm;codecs=opus content type (Chrome default). Multer audioFilter accepts it via the startsWith check.	Edge Case
9	no audio file attached returns 400	No multipart body. Controller detects req.file is undefined and returns 400 'No audio file provided'.	Error Handling
10	mock prediction returned when image missing	DB returns a case row with image_path:null. Controller cannot call Python and falls back to generateMockPrediction().	Success Story
11	mock prediction contains required fields	Verifies the fallback object has predicted_condition, confidence, and all_scores keys so the frontend can always render results.	Success Story
12	no image linked to case returns 400	Case exists in DB but image_path is null. Expects 400 'No image uploaded for this case'.	Error Handling
13	successful inference stores result in DB	Mocks fetch to return a valid prediction from Python. Controller stores it in DB and returns 200 with the prediction object.	Success Story
14	inference stores heatmap_url from Python	Python response includes heatmap_path. Controller constructs the full URL and stores it. Confirms it appears in the response.	Success Story
15	inference fallback on Python 500 response	fetch returns HTTP 500 from Python. Controller catches it and returns a mock prediction with a console warning rather than crashing.	Edge Case
16	GET results includes heatmap_url when present	DB row has a heatmap_url value. Expects 200 with heatmap_url populated so the frontend can render the Grad-CAM overlay.	Success Story
17	GET results with null heatmap_url	DB row has heatmap_url:null. Expects 200 with null value so the frontend can hide the overlay gracefully.	Success Story

#	Test Name	Description	Category
18	GET results joins symptoms array	DB row stores symptoms as an array. Confirms the response joins them into a readable string for display.	Success Story
19	case not found returns 404	DB returns zero rows for the caseld. Expects 404 Not Found.	Error Handling
20	DB error on results returns 500	pg.Pool.query throws on the SELECT. Expects 500 from the global error handler.	Error Handling
21	GET history returns populated array	DB returns multiple rows. Expects 200 with an array matching the mocked rows count and structure.	Success Story
22	GET history returns empty array when no cases	DB returns zero rows. Expects 200 with an empty array, not 404, so the frontend can show an empty-state message.	Success Story
23	GET history without auth token returns 401	No Authorization header. authenticate middleware blocks the request.	Error Handling
24	DB error on history returns 500	pg.Pool.query throws during the SELECT. The global error handler returns 500.	Error Handling
25	voice endpoint updates case transcript in DB	ASR mock returns transcript text. Controller updates the case row and returns 200 with the transcript value.	Success Story
26	inference endpoint requires authenticated user	No token on POST /:caseld/inference. authenticate middleware returns 401.	Error Handling
27	results endpoint requires authenticated user	No token on GET /:caseld/results. authenticate middleware returns 401.	Error Handling

CLIENT — React Frontend (Vitest + React Testing Library)

useScreening.test.jsx — 21 Tests · Screening Hook State Management

#	Test Name	Description	Category
1	createCase sets caseld in state	api.post resolves with {caseld}. After the call, hook state caseld matches the returned value exactly.	Success Story
2	createCase sets isLoading true then false	isLoading is true while the promise is pending and false after it resolves, so the UI can show and hide a spinner correctly.	Success Story
3	createCase clears previous error on retry	First call rejects setting error state. Second call resolves. After success, error is null, confirming error is cleared on retry.	Success Story
4	createCase failure sets error in state	api.post rejects with a network error. Expects error state to be populated with the error message string.	Error Handling
5	uploadImage posts FormData to correct URL	Checks api.post is called with a URL containing 'upload-image' and a FormData instance as the body.	Success Story
6	uploadImage sets isLoading during request	Verifies isLoading is true while the upload request is in-flight, enabling UI feedback.	Success Story
7	uploadImage failure sets error	api.post rejects. Expects error state set with the upload-failed message.	Error Handling
8	submitVoice skips upload when blob is null	Calls submitVoice(null). api.post must not be called, confirming voice recording is optional in the screening flow.	Edge Case
9	submitVoice uses .wav extension for WAV blob	MediaRecorder returns audio/wav blob. Checks the FormData filename ends with .wav before posting.	Edge Case
10	submitVoice uses .webm extension for Chrome	MediaRecorder returns audio/webm blob. Checks filename ends with .webm to match Chrome's recording format.	Edge Case
11	submitVoice uses .ogg extension for OGG	audio/ogg blob from MediaRecorder. Checks filename ends with .ogg.	Edge Case
12	submitVoice uses .mp4 extension for Safari	audio/mp4 blob. Checks filename ends with .mp4 for Safari compatibility.	Edge Case
13	runInference returns and stores prediction	api.post resolves with a prediction object. Hook stores it in results state for the ResultsPage to consume.	Success Story
14	runInference failure sets error	api.post rejects. error state is set with the inference-failed message.	Error Handling
15	getResults stores full result in state	api.get resolves with complete results object. Hook stores it and clears any previous error.	Success Story
16	getResults failure sets error	api.get rejects. error state is updated with the fetch-failed message.	Error Handling
17	getHistory calls correct GET endpoint	Checks api.get is called with the /history/list URL, confirming the hook uses the right path.	Success Story
18	getHistory failure sets error	api.get rejects. error state is updated.	Error Handling
19	reset clears caseld to null	After createCase succeeds and sets caseld, reset() returns caseld to null.	Success Story
20	reset clears results to null	After getResults succeeds and populates results, reset() returns results to null.	Success Story
21	reset clears error to null	After any failure sets the error state, reset() returns error to null.	Success Story

useVoiceRecorder.test.jsx — 15 Tests · Microphone & Recording Hook

#	Test Name	Description	Category
1	startRecording sets isRecording to true	getUserMedia resolves successfully. After startRecording(), isRecording state is true and a MediaRecorder instance is active.	Success Story
2	mic permission denied sets error message	getUserMedia rejects with NotAllowedError. isRecording stays false and error state contains a user-friendly permission-denied message.	Error Handling
3	calling startRecording twice is a no-op	Calls startRecording twice in a row. The second call returns early without creating a duplicate MediaRecorder instance.	Edge Case
4	stopRecording sets isRecording to false	After start then stop, isRecording is false, confirming the recording state machine transitions correctly.	Success Story
5	stopRecording produces an audio Blob	After recording and stopping, audioBlob in state is a Blob instance with a non-zero size.	Success Story
6	stopRecording when idle is a no-op	Calls stop without a prior start. No error is thrown and all state values remain unchanged.	Edge Case
7	clearRecording resets blob to null	After a recording, clearRecording() sets audioBlob to null so the user can record again.	Success Story
8	clearRecording resets duration to 0	clearRecording() resets the duration counter to 0.	Success Story
9	duration increments each second	Uses fake timers to advance 3 seconds. duration state equals 3, confirming the interval timer fires correctly.	Success Story
10	recording auto-stops at maxDuration	Advances fake timer past maxDuration (default 60 s). Recording stops automatically and isRecording becomes false.	Edge Case
11	timer stops incrementing after stopRecording	Manually stops recording then advances the timer further. duration does not increment after stop.	Edge Case
12	WAV conversion succeeds	AudioContext.decodeAudioData resolves with audio data. audioBlob is set to the converted WAV Blob.	Success Story
13	WAV fallback when decodeAudioData rejects	decodeAudioData rejects. Hook falls back to the raw MediaRecorder blob rather than throwing, ensuring audio is never lost.	Error Handling
14	mic device not found sets error	getUserMedia rejects with NotFoundError. error state contains a message indicating no microphone was detected.	Error Handling
15	recording duration does not go below zero	Calls stopRecording before the first timer tick. duration stays at 0, confirming it never goes negative.	Edge Case

imageValidation.test.js — 23 Tests · File Validation & Confidence Helpers

#	Test Name	Description	Category
1	JPEG file passes validation	Creates a File with type image/jpeg under 10 MB. validateImageFile returns {valid:true, errors:[]}	Success Story
2	PNG file passes validation	Creates a File with type image/png under 10 MB. Returns {valid:true, errors:[]}	Success Story
3	GIF file fails validation	Creates a File with type image/gif. Returns {valid:false} with an error message about unsupported file type.	Error Handling
4	file over 10 MB fails validation	Creates a File with size 10 MB + 1 byte. Returns {valid:false} with an error message about file size.	Error Handling
5	GIF over 10 MB returns both errors	Creates an oversized GIF. Returns {valid:false} with both a type error and a size error in the errors array.	Error Handling
6	400x400 image passes dimension check	Mocks an Image that fires onload with width=400, height=400. Callback receives an empty errors array.	Success Story

#	Test Name	Description	Category
7	300x300 image passes at minimum boundary	Mocks width=height=300 (minimum allowed). Callback receives no errors, confirming the lower boundary is inclusive.	Success Story
8	200x150 image fails dimension check	Below minimum size. Callback receives an error string confirming the image is too small.	Error Handling
9	error message includes actual dimensions	Verifies the error string reports '200x150' so the user knows exactly what dimensions were rejected.	Error Handling
10	onerror path returns graceful error	Mocks Image.onerror to fire. Callback receives a 'could not load image' error without throwing an unhandled exception.	Edge Case
11	very large image accepted if dimensions OK	Mocks a 4000x4000 image. No error is returned because only minimum dimensions are enforced, not maximum.	Edge Case
12	non-square image passes if both sides above min	Mocks 600x400. Both dimensions are above the minimum so no error is returned.	Success Story
13	score 0.95 returns high	Passes 0.95 to getConfidenceLevel. Returns 'high'.	Success Story
14	score 0.8 returns high at exact boundary	Passes exactly 0.8. Returns 'high', confirming the boundary is ≥ 0.8 not > 0.8 .	Edge Case
15	score 0.75 returns medium	Passes 0.75. Returns 'medium'.	Success Story
16	score 0.6 returns medium at exact boundary	Passes exactly 0.6. Returns 'medium', confirming the boundary is ≥ 0.6 not > 0.6 .	Edge Case
17	score 0.3 returns low	Passes 0.3. Returns 'low'.	Success Story
18	score 0.59 returns low just below medium	Passes 0.59. Returns 'low', confirming values strictly below 0.6 are low.	Edge Case
19	high confidence returns a color string	getConfidenceColor(0.9) returns a non-empty string. Confirms the function returns something renderable for the UI.	Success Story
20	medium confidence returns a color string	getConfidenceColor(0.7) returns a non-empty string different from the high confidence color.	Success Story
21	low confidence returns a color string	getConfidenceColor(0.2) returns a non-empty string different from both high and medium colors.	Success Story
22	high and low confidence return different colors	Calls getConfidenceColor for 0.9 and 0.2. The two returned values are not equal, confirming distinct visual feedback.	Edge Case
23	medium and high confidence return different colors	Calls getConfidenceColor for 0.7 and 0.9. The two values differ, confirming three distinct color levels are applied.	Edge Case

PYTHON — Inference Service (pytest + FastAPI TestClient)

test_predict.py — 19 Tests · /health + /predict Endpoints

#	Test Name	Description	Category
1	GET /health returns 200	Calls GET /health. Expects status 200 confirming the inference service is running and the route is registered.	Success Story
2	/health lists all 7 skin condition classes	Parses the classes array and asserts all seven conditions are present: Vitiligo, Melasma, Psoriasis, Eczema, Tinea, Contact Dermatitis, Seborrheic Dermatitis.	Success Story
3	/health shows ASR model as not loaded	ASR_MODEL_PATH points to a non-existent directory. health response shows asr_model:'not loaded' so operators can detect the missing model at a glance.	Edge Case
4	POST /predict with PNG returns 200	Sends a valid 10x10 PNG image file. Expects 200 with a JSON body containing prediction data.	Success Story
5	POST /predict with JPEG returns 200	Sends a valid JPEG image. Expects 200, confirming both common image formats are accepted by the endpoint.	Success Story
6	response contains condition name key	Asserts the response JSON has a key for the top predicted skin condition so the frontend can display the diagnosis.	Success Story
7	response contains a numeric confidence score	Asserts a numeric confidence value is present in the response body for the top predicted class.	Success Story
8	confidence score is within 0.0 to 1.0	Softmax outputs must be probabilities. Asserts $0.0 \leq \text{confidence} \leq 1.0$.	Success Story
9	all_scores values sum to approximately 1.0	Sums all values in the all_scores dict. Asserts the total is within ± 0.05 of 1.0 (floating-point softmax tolerance).	Success Story
10	heatmap PNG is written to disk after predict	Checks that a UUID-named PNG file exists in HEATMAP_DIR after a successful /predict call, confirming Grad-CAM output is persisted.	Success Story
11	response contains heatmap filename key	Asserts heatmap_path key is in the response so the Express backend can construct the full heatmap URL for the client.	Success Story
12	non-image file returns 400	Sends a plain text file as the image. PIL.Image.open raises an error. Expects 400 Bad Request.	Error Handling
13	empty zero-byte file returns 400	Sends a zero-byte payload. PIL cannot open an empty buffer. Expects 400.	Error Handling
14	2048x2048 image is resized and returns 200	Sends a 2048x2048 PNG. Server resizes to 224x224 internally. Expects 200, confirming the resize prevents memory issues on large uploads.	Edge Case
15	response keys are consistent on repeated calls	Sends the same image twice and asserts the set of response keys is identical both times, confirming a stable API contract.	Edge Case
16	all_scores contains exactly 7 class keys	Asserts all_scores dict has exactly 7 keys matching the known condition names, confirming no class is dropped or added.	Success Story
17	predicted condition is one of the 7 classes	Asserts the top condition value appears in the known set {Vitiligo, Melasma...}, preventing unexpected class name strings.	Success Story
18	inference_time_ms key is present	Asserts inference_time_ms is in the response for performance monitoring and SLA tracking.	Success Story
19	wrong form field name returns 422	Sends file under field name 'file' instead of 'image'. FastAPI returns 422 Unprocessable Entity for the missing required field.	Edge Case

test_transcribe.py — 9 Tests · /transcribe Endpoint (ASR)

#	Test Name	Description	Category
1	WAV file returns 503 when ASR not loaded	ASR model directory is missing so asr_pipe is None. Sending a WAV file returns 503 Service Unavailable.	Error Handling
2	WebM file returns 503 when ASR not loaded	Same scenario with WebM audio format. Confirms 503 is returned regardless of audio format when the model is absent.	Error Handling
3	MP4 file returns 503 when ASR not loaded	Same scenario with MP4 audio. Confirms format-independent 503 response when ASR model is unavailable.	Error Handling
4	empty audio file returns 400 or 503	Zero-byte audio file. Either the file is rejected as invalid (400) or the model-unavailable path fires (503). Never 500.	Error Handling
5	503 response body is valid JSON	Asserts Content-Type is application/json so the Express backend can parse the error body without crashing.	Edge Case
6	503 error message mentions model not loaded	Asserts the response error message includes 'not loaded' (or similar) so operators know to run download_asr_model.py.	Edge Case
7	POST /transcribe with no file returns 422	Sends POST /transcribe with no multipart body. FastAPI returns 422 Unprocessable Entity for the missing required field.	Error Handling
8	very large filename still returns 503 not 500	Audio file with a 300-character filename. Server handles it without crashing and still returns 503 (model unavailable).	Edge Case
9	no filename still returns 503 or 422	Audio sent without a filename attribute in the multipart form. Server returns 503 or 422 but never an unhandled 500.	Edge Case

test_heatmap.py — 10 Tests · /heatmaps Endpoint & Path Security

#	Test Name	Description	Category
1	GET /heatmaps/{file} returns 200	Writes a real PNG to the heatmap directory then requests it by name. Expects 200 OK.	Success Story
2	response Content-Type is image/png	Checks the Content-Type header contains 'image/png' so browsers can display it as an image without a download prompt.	Success Story
3	response body has valid PNG magic bytes	Checks the first 8 bytes are the PNG signature confirming the file is a genuine PNG, not corrupted data.	Success Story
4	response body is non-empty	Asserts len(response.content) > 0 confirming a real file was served, not an empty 200 response.	Success Story
5	missing file returns 404	Requests a filename that was never written to the heatmap directory. Expects 404 Not Found.	Error Handling
6	empty filename segment returns 404 or 405	GET /heatmaps/ with no filename. Expects 404 (no file) or 405 (no route match), not a crash.	Error Handling
7	path traversal attempt is blocked	Requests ../../etc/passwd. Server resolves the path, detects it escapes the heatmaps directory, and returns 404.	Edge Case
8	URL-encoded slash traversal is blocked	Uses %2F (URL-encoded forward slash) in the traversal string. Server decodes, checks the resolved path, and returns 404.	Edge Case
9	XSS string in filename returns 404 not 500	Sends a script tag as the filename. Server returns 404 without an unhandled exception, confirming no template injection.	Edge Case
10	500-char filename returns 404 not 500	Very long filename string. Filesystem stat fails gracefully; server returns 404, not a 500 crash.	Edge Case

test_utils.py — 17 Tests · Audio Loader, GradCAM, Heatmap Overlay

#	Test Name	Description	Category
1	load_audio_wav returns float32 1-D numpy array	load_audio_wav on a valid WAV file returns a numpy ndarray with dtype float32 and ndim == 1.	Success Story
2	correct sample count for a 1-second clip	A 1-second 16 kHz WAV should return an array of length approximately 16000. Asserts within a small tolerance.	Success Story
3	silent WAV produces an all-zero array	Silent WAV file (zero-amplitude samples). All values in the returned array are 0.0.	Success Story
4	output is always 1-D mono not stereo	Confirms ndim == 1. Stereo files must be mixed to mono by the loader before returning.	Success Story
5	short 0.1-second clip handled without error	Very short WAV (0.1 s). Function returns an array of the correct length without raising any exception.	Success Story
6	non-existent file path raises an exception	Passes a path that does not exist. Confirms the function raises FileNotFoundError rather than returning None.	Error Handling
7	GradCAM output is a 2-D numpy array	GradCAM.generate returns an ndarray with ndim == 2, representing the spatial activation map.	Success Story
8	GradCAM values are all within 0.0 to 1.0	After normalisation, every value in the CAM array is >= 0.0 and <= 1.0.	Success Story
9	class_idx used is within valid range 0 to 6	The class index selected for backpropagation is between 0 and 6 inclusive (7 conditions).	Success Story
10	explicit class_idx is respected by GradCAM	Passes class_idx=3 explicitly. Asserts GradCAM uses that index rather than choosing the argmax of the model output.	Edge Case
11	heatmap overlay output is a numpy ndarray	create_heatmap_overlay returns a numpy array, not a PIL Image or raw bytes object.	Success Story
12	overlay has 3 channels RGB	shape[-1] == 3, confirming the JET colormap is applied as RGB rather than RGBA.	Success Story
13	overlay shape matches IMG_SIZE 224x224	Height and width equal IMG_SIZE (224) so the heatmap aligns perfectly with the original 224x224 input image.	Success Story
14	overlay pixel dtype is uint8	dtype == uint8, confirming values are in the 0-255 range required for PNG encoding.	Success Story
15	all pixel values are in range 0 to 255	min(overlay) >= 0 and max(overlay) <= 255.	Success Story
16	all-zero CAM produces a valid overlay	CAM of all zeros (no activation anywhere). Overlay still returns a valid uint8 array without divide-by-zero errors.	Edge Case
17	all-one CAM produces a valid overlay	CAM of all ones (maximum activation everywhere). Overlay is still a valid array confirming the full colour range is handled.	Edge Case